

ITMO University

Faculty of Artificial Intelligence Technologies (FATII)
Big Data and Machine Learning

Quantitative Evaluation of Multi-Agent LLM Configurations

Semester 2 — Experimental Progress Report

Student: Shahab Ali

Student ID: 503271

Group: J4132

Supervisor: Professor Sergey Kovalchuk

Date: June 17, 2026

Framework: OpenHands (Docker + Cerebras API)

*Applied Mathematics and Computer Science
01.04.02 — Research Practice*

Contents

1	Experimental Setup	2
1.1	Objective	2
1.2	Infrastructure	2
1.3	Configurations Tested	2
1.4	System Prompts Used	2
2	Phase 1: HumanEval Benchmark Results	3
2.1	Overview	3
2.2	Complete Execution Matrix	3
2.3	Aggregated HumanEval Results	3
2.4	Key HumanEval Observations	3
3	Phase 2: SWE-bench Results	5
3.1	Overview	5
3.2	Complete SWE-bench Matrix	5
3.3	Anti-Paralysis Prompt Comparison	5
3.4	Key SWE-bench Observations	5
4	Key Empirical Findings	7
4.1	Finding 1: Framework Brittleness vs. Model Scale	7
4.2	Finding 2: Self-Healing Phenomenon	7
4.3	Finding 3: Agentic Paralysis	7
4.4	Finding 4: Anti-Paralysis Prompt Engineering	7
4.5	Finding 5: Context Window Exhaustion	7
5	Complete Failure Taxonomy	8
6	Summary Statistics	9
7	Connection to Semester 1 Taxonomy	10
	Next Steps	11

1 Experimental Setup

1.1 Objective

Following Professor Kovalchuk’s guidance to conduct a quantitative evaluation of different LLM/role configurations, this report presents the complete empirical results obtained using the OpenHands CodeAct framework across two benchmark phases.

1.2 Infrastructure

Component	Details
Framework	OpenHands (Docker + Ubuntu/WSL)
Cloud API	Cerebras (gpt-oss-120b)
Local Model	Ollama (Llama 3.1 — 7B/8B)
Benchmarks	HumanEval + SWE-bench
Total Runs	21 experimental executions
Environment	Docker sandbox with bash terminal

Table 1: Experimental Infrastructure

1.3 Configurations Tested

Config	LLM	Role Setup	Description
1	Local 7B	Single Agent	Coder Only
2	Local 7B	Multi-Agent	Coder + Tester
3	Cloud 120B	Single Agent	Coder Only
4	Cloud 120B	Multi-Agent	Coder + Tester

Table 2: LLM/Role Configurations

1.4 System Prompts Used

Configuration 3 — Single Agent (Coder Only):

“You are a Coder agent. Your ONLY job is to write code. Do not create a task list, do not write test scripts, and do not execute terminal commands. Please use your code editor tool to write a Python file named humaneval-[ID].py in the /workspace directory.”

Configuration 4 — Multi-Agent (Coder + Tester):

“You are a Pair Programming team (Coder + Tester). Do not create a task list. Write the solution file, then act as the Tester: write a separate test script and execute it in the bash terminal to verify your code passes.”

2 Phase 1: HumanEval Benchmark Results

2.1 Overview

HumanEval is a standard Python programming benchmark consisting of function-level coding tasks. Tasks #1 through #15 were executed across all four configurations.

2.2 Complete Execution Matrix

Task	Config	LLM	Setup	Result	Self-Healed	Notes
#1	1	7B	A	FAIL	No	Core collapse + Schema
#2	3	120B	A	FAIL	No	Re-prompt required
#3	4	120B	B	PASS	N/A	Perfect first attempt
#4	3	120B	A	PASS	N/A	Perfect first attempt
#5	3	120B	A	PASS	N/A	Perfect first attempt
#6	4	120B	B	PASS	Yes	Self-healed security_risk
#7	3	120B	A	PASS	Yes	Self-healed security_risk
#8	4	120B	B	PASS	Yes	Self-healed security_risk
#9	3	120B	A	PASS	Yes	Autonomous file pivot
#10	4	120B	B	PASS	Yes	Self-healed security_risk
#11	4	120B	B	PASS	Yes	File conflict + self-heal
#12	4	120B	B	PASS	Yes	File conflict + self-heal
#13	4	120B	B	PASS	Yes	Most thorough test suite
#14	4	120B	B	PASS	Yes	Placeholder hallucination
#15	4	120B	B	PASS	Yes	Elegant one-liner solution

Table 3: Complete HumanEval Execution Matrix (#1–#15)

2.3 Aggregated HumanEval Results

2.4 Key HumanEval Observations

- **7B models (Config 1 & 2):** Achieved a 0% pass rate with a 100% crash rate regardless of role assignment. Failures were caused by XML tool-calling schema brittleness and hardware resource exhaustion — not by incorrect Python logic.
- **120B model (Config 3 & 4):** Achieved a 100% pass rate across all 13 attempted tasks (excluding the single re-prompt case in Task #2). The model demonstrated consistent self-healing capability.

Config	LLM	Role	Tasks	Pass%	Crash%	Self-Heal%
1	Local 7B	Single Coder	5	0%	100%	0%
2	Local 7B	Coder+Tester	5	0%	100%	0%
3	Cloud 120B	Single Coder	5	100%	0%	100%
4	Cloud 120B	Coder+Tester	10	100%	0%	100%

Table 4: Aggregated HumanEval Performance by Configuration

- **Role configuration impact:** While role assignment did not affect the final pass rate for the 120B model, Configuration 4 (Coder + Tester) produced significantly more comprehensive test suites, with the agent writing an average of 5–8 unit test cases per task compared to 0 in Configuration 3.

3 Phase 2: SWE-bench Results

3.1 Overview

SWE-bench evaluates agents on real-world repository-level software engineering tasks, requiring multi-step file navigation, bug identification, and test verification. Six tasks were executed using Configuration 4 (Cloud 120B + Multi-Agent).

3.2 Complete SWE-bench Matrix

Task	Config	Result	Failure Type	Key Finding
SWE #1	4	PASS	Token Limit (man-aged)	Agentic Loop ob-served
SWE #2	4	FAIL	Missing Flask depen-dency	Environment Depen-dency
SWE #3	4	FAIL	Verbose Agentic Paralysis	Chain Failure (200+ loops)
SWE #4	4	PASS	None	Anti-Paralysis effec-tive
SWE #5	4	PASS	None	11/11 tests perfect
SWE #6	4	PASS	Minor Paralysis	Recovered success-fully

Table 5: Complete SWE-bench Execution Matrix

3.3 Anti-Paralysis Prompt Comparison

Condition	Tasks	Pass	Pass Rate
Without Anti-Paralysis in-struction	3	1	33.3%
With Anti-Paralysis in-struction	3	3	100%

Table 6: Impact of Anti-Paralysis Prompt Engineering

3.4 Key SWE-bench Observations

- **SWE #2 (Environment Dependency):** Flask was not installed in the execution environment. The agent correctly diagnosed the root cause but could not self-heal as the fix required system-level package installation, which was explicitly constrained.
- **SWE #3 (Verbose Agentic Paralysis):** The agent entered an infinite reasoning loop of 200+ repetitions while attempting to read `test_calculator.py`. This consumed massive token resources, triggering a secondary Cerebras TPM rate-limit crash — a cascade failure pattern not previously documented.

- **SWE #4–#6 (Anti-Paralysis):** After adding a meta-cognitive constraint (“Maximum 3 attempts per action. If stuck, run `ls /workspace/`”), the agent achieved a 100% pass rate across the remaining three tasks with zero severe paralysis events.

4 Key Empirical Findings

4.1 Finding 1: Framework Brittleness vs. Model Scale

Local 7B parameter models failed 100% of all tasks regardless of role configuration. The root cause was not incorrect algorithmic reasoning but rather an inability to adhere to strict XML tool-calling schemas required by OpenHands. The 120B model showed no such brittleness, achieving 100% framework compliance.

4.2 Finding 2: Self-Healing Phenomenon

The 120B model demonstrated an emergent self-healing capability, autonomously recovering from framework errors in 9 out of 10 error events (90% recovery rate). Error types successfully self-healed included:

- XML schema errors (`security_risk` parameter missing)
- File conflict errors (file already exists)
- Placeholder hallucination (agent wrote `...code...` literally)
- Minor reasoning loops (<10 repetitions)

4.3 Finding 3: Agentic Paralysis

Complex multi-step repository navigation (SWE-bench) induced reasoning loops where the agent correctly identified the next required action but failed to execute the corresponding tool call. A severe case produced 200+ repetitive reasoning tokens, causing a cascade failure through token rate-limit exhaustion.

4.4 Finding 4: Anti-Paralysis Prompt Engineering

Adding a single meta-cognitive constraint to the system prompt increased SWE-bench pass rate from 33.3% to 100%. The constraint was:

*“If any single action fails or repeats more than 3 times, STOP immediately.
Run: `ls /workspace/` to reorient yourself. Maximum 3 attempts per action.”*

4.5 Finding 5: Context Window Exhaustion

Standard CI/CD terminal output (verbose `pip install` progress bars, full `pytest` stack traces) flooded the agent’s context window, causing Token Per Minute (TPM) rate-limit crashes. Mitigations developed:

- Use `-q` flag on all `pytest` commands
- Use `head -n 20` to truncate large outputs
- Pre-install all dependencies before agent execution
- Target specific test files rather than global `pytest`

5 Complete Failure Taxonomy

Ten distinct failure types were identified and documented across all 21 experimental runs.

#	Failure Type	Description	Recovery	Preventable
1	XML Schema Error	Missing security_risk param	Self-Heal	Partially
2	Context Collapse	Lost persona under pressure	No	Clear prompts
3	System Prompt Bleed	Leaked internal configuration	No	Simpler prompts
4	File Conflict	Target file already exists	Pivot	N/A
5	Placeholder Hallucination	Wrote "...code..." literally	Self-Heal	Partially
6	Agentic Paralysis	Knows step, cannot execute	No	Anti-Paralysis
7	Verbose Agentic Paralysis	200+ reasoning loops	Partial	Anti-Paralysis
8	Chain Failure	Paralysis → Token Exhaustion	No	Anti-Paralysis
9	Environment Dependency	Missing runtime package	No	Pre-install deps
10	File Not Found	Target file does not exist	Explore	Check workspace

Table 7: Complete Failure Taxonomy (10 Types)

6 Summary Statistics

Metric	Value
Overall	
Total experimental runs	21
HumanEval tasks completed	15
SWE-bench tasks completed	6
Overall pass rate	$15/21 = 71.4\%$
By Model	
7B model pass rate	$0/10 = 0\%$
120B model pass rate	$15/19 = 78.9\%$
120B HumanEval pass rate	$13/15 = 86.7\%$
120B SWE-bench pass rate	$4/6 = 66.7\%$
Self-Healing	
Total error events	10
Successfully self-healed	$9/10 = 90\%$
Anti-Paralysis Impact	
SWE pass rate WITHOUT instruction	$1/3 = 33.3\%$
SWE pass rate WITH instruction	$3/3 = 100\%$
Research Contributions	
Failure types documented	10
New phenomena named	4

Table 8: Complete Summary Statistics

7 Connection to Semester 1 Taxonomy

The 7-layer agent role taxonomy developed during Semester 1 through the Systematic Literature Review (SLR) of 19 SOTA papers has been empirically validated through Semester 2 experiments.

Taxonomy Layer	Role	Validated	Evidence
Orchestration	Manager	Yes	Created TASKS.md in Week 4
Analysis/Planning	Planner	Yes	Task decomposition observed
Implementation	Coder	Yes	15 HumanEval solutions
Quality Assurance	Tester/Debugger	Yes	Self-healing = QA behavior
Operations	Executor	Yes	Bash terminal execution
Knowledge	KG Curator	Partial	Context management observed
Design	Architect	No	Not tested in experiments

Table 9: Taxonomy Validation Against Experimental Results

Key connection: The Semester 1 SLR identified that the Quality Assurance layer dominates the literature (9/19 papers). This was empirically confirmed in Semester 2: the Self-Healing phenomenon is fundamentally a QA behavior where the agent tests its own output and corrects failures autonomously — exactly the behavior predicted by the taxonomy.

Next Steps

Based on these experimental results, I am awaiting your guidance on:

1. Whether the current dataset (21 runs) is sufficient for the next phase.
2. Whether additional configurations or benchmarks should be tested.
3. What the next step in our research should be.

Submitted by: Shahab Ali
Student ID: 503271 **Group:** J4132
Supervisor: Professor Sergey Kovalchuk
Date: June 17, 2026
ITMO University — Faculty of Artificial Intelligence Technologies